TECHNICAL IMPLEMENTATION BRIEF

Annex A to Research Proposal: Geometric Coherence as a Safety Criterion for AGI

To: Chris Tucker, cartheur, Volatco

From: Daniel Solis, DUBITO Inc. - Dubito Ergo AGI Safety Project (solis@dubito-ergo.com)

Date: April 2026

Re: Quasiperiodic Architecture Overlay on GA144 - Rationale, Feasibility, and Implementation Sketch

Purpose of this document

This brief translates the theoretical and empirical findings of the CQFT/PFT framework into a concrete implementation specification for the Volatco GA144 platform. It addresses three questions: (1) why a quasiperiodic routing topology is the correct choice for AGI field dynamics; (2) how to implement it on existing hardware without silicon modification; and (3) what to build first, and in what order, to produce a publishable hardware result within three months.

1. Why Quasiperiodic? The Geometric Ordering Result

The CQFT simulation has now been run at system sizes N = 200 through N = 1,000, across three substrate geometries, with a total of 20+ runs per condition at the largest scale. The geometry-ordering result is unambiguous in every observable:

Observable	Square lattice	Random graph	Penrose quasicrystal
Embedding gap Γ	0.87 (large)	0.19 (moderate)	0.20 (stable, small)
Prigogine regime	Frustrated far-from-equil.	Near-equilibrium	Productive far-from-equil.
Dubito □	0.09 (overworking)	≈0 (absorbed)	0.08 (efficient)
Spectral drift	Intermediate	≈0 (frozen)	Highest (geometrically alive)
D_c stability across N	Non-monotone (worst)	Monotone growth	Stable [1.41, 1.49] (best)
Order ΔR with λ > 0	Not measured (frustrated)	---	+0.1657 (165.7× SNR)

The square lattice - the physical topology of the GA144 18×8 grid - is the worst-performing geometry by every metric.

This is not a criticism of the hardware. It is a measurement. The rectangular mesh imposes a phase-frustration structure that is precisely what the CQFT field finds most difficult to resolve. A quasiperiodic routing overlay corrects this at the software level, with no silicon change required.

1.1 Why the lattice frustrates the field

In the Kuramoto-like phase dynamics, nodes on a square lattice experience a geometric frustration: their four nearest neighbors are arranged at 0°, 90°, 180°, 270°. This enforces a four-fold symmetry preference on the phase landscape that conflicts with the field's natural tendency toward quasiperiodic self-organization. The result is the non-monotone D_c scaling observed experimentally, the lattice resists the field's self-consistency iteration.

Penrose geometry (and its 3D analogue, the icosahedral quasicrystal / Poincaré Dodecahedral Space) avoids this frustration. Nodes have 5- or 10-fold local environments that do not impose a conflicting symmetry. The field finds a self-consistent configuration with less "work" - lower effective dubito fraction at the productive operating point, lower Γ, and higher spectral drift indicating geometric aliveness.

1.2 The 3D case: Poincaré Dodecahedral Space (PDS)

The 3D extension to PDS adds something the 2D Penrose tiling does not have: topological protection. PDS is constructed by identifying opposite faces of a dodecahedron with a 36° twist. Its fundamental group π_1 (PDS) is the binary icosahedral group of order 120, a non-trivial, non-abelian group.

What this means physically: global coherence modes on PDS are topologically protected against local perturbations. A phase pattern that winds around the manifold cannot be deformed to zero by local noise. For AGI dynamics, this translates to a substrate that actively stabilizes the field's global self-consistency, the geometric analogue of error correction. The 3D icosahedral simulation results (D_c = 1.41–1.47 stable, highest iso score, lowest Γ at N = 500) are consistent with this topological protection hypothesis. Full validation requires N ≥ 500 in 3D with multiple independent runs, which is a work package in the proposed programme.

---

2. Hardware Feasibility on GA144

2.1 Memory budget per core: it fits

The critical question for GA144 feasibility is whether the CQFT field state fits within each core's 64-word local RAM. The analysis is as follows:

Variable	Storage	Notes
Phase θ_i	1 word (18-bit)	Scaled: 0 → 65535 maps to 0 → 2π
Amplitude A_i	1 word	Fixed-point [0, 3] × 10000
Memory kernel buffer (exp. approx)	8 words	Short deque: 8 past values + weights
Logical neighbor IDs (k=7)	7 words	One word per neighbor node ID
Coupling weights W_ij (k=7)	7 words	Pre-scaled exponential; static or updated
Routing table (multi-hop)	8 words	Next-hop physical address per logical neighbor
Phase receive buffer	7 words	Last received phase per logical neighbor
Accumulator / coupling sum	2 words	Running sum for phase and amplitude updates
Dubito accumulators (2×)	2 words	pred-gradient + total-gradient running sums
Working registers	8 words	Forth stack scratch; loop counters; flags
TOTAL	51 words	Fits within 64-word limit with 13 words spare

Memory verdict: feasible with k = 7 logical neighbors and an 8-step memory kernel.

The 13 spare words allow for one additional observable (e.g., a local Γ proxy counter) or a longer kernel buffer if kernel length is reduced to 6 steps.

2.2 Message format: packing field state into 18-bit words

Each inter-core message must carry sufficient field state for the receiving node to update its coupling. The minimum viable message is phase only (1 word). An extended message carries phase + amplitude:

```
18-bit message format (extended, 2-word message):

Word 1:  [ NODE_ID (8 bits) | PHASE_HI (10 bits) ]
Word 2:  [ PHASE_LO (8 bits) | AMPLITUDE (10 bits) ]

PHASE:     18-bit value, range 0-262143, maps to 0-2π (14-bit resolution:
~0.022°)
AMPLITUDE: 10-bit value, range 0-1023, maps to 0.0-3.0 (fixed-point × 341)
NODE_ID:   8-bit logical node index (supports N ≤ 256 logical sites)

Minimum message (phase only): 1 word - use for fast inner-loop coupling
Extended message (phase+amp):  2 words - use for geometry update step only
```

2.3 Routing strategy: software-defined quasiperiodic topology

The GA144 physical mesh supports only nearest-neighbor communication. Quasiperiodic connectivity requires logical links of varying lengths. The implementation uses a static multi-hop routing table loaded from SPI flash at boot:

- Each logical Penrose node i has k = 7 logical neighbors $j_1 \ldots j_7$, determined offline from the cut-and-project construction.
- For each logical neighbor j, the routing table stores the physical next-hop core address on the path from i to j.
- Short-range logical links (distance ≤ 2 physical hops): direct message, latency ≈1 μs.
- Long-range logical links (distance 3-6 physical hops): forwarded through intermediate cores acting as routers. Dedicated routing cores (4-6 of the 144) handle forwarding only.
- Hybrid mode: use dense local physical neighbors for high-frequency phase coupling; sparse routed logical links for the slow geometry update step only. This minimizes routing overhead on the critical path.

Approach	Latency	Energy cost	Best for
Direct physical neighbor	~10 ns	Minimal	Fast phase coupling loop
2-hop routed logical link	~100 ns	2×	Medium-range quasiperiodic coupling
4-hop routed logical link	~500 ns	4×	Long-range quasiperiodic coupling
Dedicated routing cores	~200 ns	Shared overhead	Full Penrose overlay, N ≤ 120

Recommendation: for the first proof-of-concept (N = 50-100 logical sites), restrict the Penrose patch to a region where all logical links span ≤ 3 physical hops. The Robinson inflation generator can produce such patches reliably. This eliminates dedicated routing cores and fits the full simulation in a single GA144 chip.

3. Implementation Sketch: Node-Level Forth Pseudocode

The following is a minimal viable implementation skeleton for a single logical CQFT node running on a GA144 core (or small core cluster). It assumes the 7-neighbor memory layout from Section 2.1 and the 2-word message format from Section 2.2. Comments use the (before -- after) Forth stack notation.

3.1 Core-level data layout

```
( === Data layout in local RAM, 64-word budget === )

VARIABLE phase          ( current phase, 18-bit scaled )
VARIABLE amplitude      ( current amplitude, 10-bit scaled )
VARIABLE pred-phase     ( memory kernel prediction )

( Memory kernel: 8-step deque of past phases )
CREATE mem-buf 8 CELLS ALLOT
VARIABLE mem-head       ( ring buffer pointer )

( Logical neighbor table: 7 neighbors )
CREATE nb-id    7 CELLS ALLOT  ( logical node IDs )
CREATE nb-route 7 CELLS ALLOT  ( physical next-hop address )
CREATE nb-wt    7 CELLS ALLOT  ( coupling weights W_ij, pre-scaled )
CREATE nb-phase 7 CELLS ALLOT  ( last received phase from each neighbor )

( Dubito watchdog accumulators )
VARIABLE pred-acc       ( prediction gradient accumulator )
VARIABLE total-acc      ( total gradient accumulator )
VARIABLE step-count     ( global step counter )
```

3.2 Phase update loop (fast path)

```
( === Phase coupling: Kuramoto-like, fast path === )

: sin-approx  ( phase-diff -- sin*1000 )
  ( 3rd-order polynomial approx. for |x| < pi/2, )
  ( lookup table otherwise. 18-bit inputs. )
  ... ;

: coupling-sum  ( -- coupling*1000 )
  0  ( accumulator )
  7 0 DO
    nb-phase I CELLS + @     ( phase_j )
    phase @  -               ( phase_j - phase_i )
    sin-approx               ( sin(phase_j - phase_i) * 1000 )
    nb-wt I CELLS + @  *     ( weight * sin(delta_phase) )
    1000 /                   ( descale )
    +                        ( accumulate )
```

```
  LOOP ;

: update-phase  ( -- )
  coupling-sum              ( coupling * 1000 )
  BETA * 1000 /             ( beta * coupling )
  noise-word +              ( + small gaussian noise )
  DT * 1000 /               ( * dt )
  phase +!                  ( phase_i += ... )
  phase @  PHASE-MAX MOD    ( wrap to [0, 2pi] )
  phase ! ;
```

3.3 Geometry update step (slow path, every N_slow cycles)

```
( === Geometry backprop: coherence-weighted displacement === )
( Runs every N_slow = 50 phase steps (slow timescale eta_x) )

: coherence-weight  ( j -- cos_weight*1000 )
  nb-phase SWAP CELLS + @  ( phase_j )
  phase @ -                ( phase_j - phase_i )
  cos-approx ;             ( cos(delta_phase) * 1000 )

: adjust-routing-weight  ( j -- )
  DUP coherence-weight    ( j cos_w )
  LAMBDA * 1000 /         ( lambda * cos_w )
  BASE-WEIGHT +           ( base + phase-driven component )
  nb-wt ROT CELLS + ! ;   ( store updated weight for neighbor j )

: geometry-step  ( -- )
  7 0 DO  I adjust-routing-weight  LOOP ;

( This implements the cross-derivative: dW_ij/dx_i via weight modulation )
( Coherent pairs get higher weights; incoherent pairs deprioritized. )
( This is lambda > 0 on physical silicon. )
```

3.4 Dubito watchdog

```
( === Dubito operator: self-prediction fraction === )

: update-pred  ( -- )
  ( Compute memory kernel prediction from mem-buf )
  0  8 0 DO
    mem-buf I CELLS + @    ( past phase at lag i )
    KERNEL-WEIGHT I *  +   ( weighted sum )
  LOOP
  8 /  pred-phase ! ;      ( normalise -> prediction )

: dubito  ( -- D*1000 )
  pred-acc @  1000 *
  total-acc @ 1 MAX  / ;   ( pred / total, scaled )

: watchdog-check  ( -- )
  dubito  DUBITO-THRESHOLD >
  IF  J7-TRIGGER  THEN ;   ( fire J7 watchdog if D > threshold )

( DUBITO-THRESHOLD = 900 (i.e., D > 0.9) triggers watchdog )
( Maps 'runaway self-reference' to a hardware-level safety signal )
```

3.5 Main node loop

```
( === Main node loop === )

: main-loop  ( -- )
  BEGIN
    receive-neighbor-phases    ( update nb-phase[] from message queue )
    update-phase               ( fast path: Kuramoto coupling + noise )
    update-amplitude           ( amplitude dynamics )
    update-mem-buf             ( push current phase to memory ring )
    update-pred                ( compute memory kernel prediction )
    broadcast-phase            ( send phase to logical neighbors )
    step-count @ N_SLOW MOD 0= ( every N_SLOW steps: )
    IF  geometry-step  THEN    (   slow path: update routing weights )
    step-count @ WATCHDOG_INT MOD 0=
    IF  watchdog-check  THEN   ( periodic dubito safety check )
    1 step-count +!            ( increment step counter )
  AGAIN ;
```

4. Recommended First Experiment

The fastest path to a publishable hardware result is the λ bifurcation test on a single GA144 chip, using a small Penrose overlay. This requires one Volatco evaluation board, approximately four weeks of implementation work, and produces a result that directly validates the most important claim of the research programme.

4.1 Step-by-step: the λ bifurcation test

1. Generate a 2D Penrose patch with N = 64 logical sites using the Robinson inflation generator (existing code). Select the patch such that all logical links span ≤ 3 physical hops on the 18×8 grid. Export: logical neighbor list, routing table (next-hop physical addresses), pre-scaled coupling weights.
2. Load routing tables into SPI flash. Write a Forth boot word that reads the routing table and populates each core's nb-route[] and nb-wt[] arrays at startup.
3. Implement the main-loop word (Section 3.5) on each core. Set N_SLOW = 50 (geometry update every 50 phase steps). Set λ = 0 first (ablation condition: geometry-step updates nothing, weights stay constant).
4. Run for 500 steps. Record order parameter R = |mean(exp(i*phase))| across all 64 nodes every 50 steps. This is the ablation baseline.
5. Set λ = 0.8 (coupling-ON condition: geometry-step updates routing weights by coherence). Run again for 500 steps. Record R trajectory.
6. Prediction: R_coupled - R_ablation ≈ +0.10 to +0.14 (extrapolating from N=1000 simulation). If confirmed on silicon, this is the hardware bifurcation result.
7. Measure power draw on J2/J3 shunt resistors at 100-step intervals. The coupled condition should exhibit a characteristic oscillation in current draw (the breathing phenomenon, overshoot fraction ≈0.12). This is a secondary result of independent value.

Expected timeline: 4 weeks to first result.

Week 1: Penrose patch generation + routing table export.

Week 2: Forth implementation of main-loop on single core; debug with polyForth terminal on J7.

Week 3: Scale to 64-core deployment; ablation run.

Week 4: Coupling-ON run; power measurement; data analysis.

Deliverable: ΔR measurement on physical hardware + power oscillation fingerprint.

4.2 Secondary experiment: Penrose vs. lattice Γ comparison

Run the same field dynamics with two routing configurations: (A) native physical nearest-neighbor lattice routing, (B) 64-node Penrose overlay. Compute a Γ proxy on the host from the recorded phase and amplitude trajectories:

- D_f proxy: box-count the amplitude field projected onto the 2D physical grid.
- D_c proxy: compute the amplitude-weighted mean pair-distance for a random subsample of 200 core pairs.
- Γ proxy: |D_f − D_c|.

Prediction: Γ_lattice / Γ_Penrose ≈ 4 (based on simulation ratio 0.87 / 0.20). This is the hardware confirmation of the geometry-ordering result.

4.3 Scaling path

Phase	N logical sites	Hardware	New capability	Timeline
Proof of concept	64	1 GA144 chip (Volatco single board)	λ bifurcation on silicon; power fingerprint	Month 1-2
Penrose vs. lattice Γ	64	Same board, two routing configs	Γ hardware measurement	Month 2-3
Extended Penrose	120	1 GA144 chip (full)	Near-full-chip quasiperiodic topology	Month 3-4
Dual-chip	250	Volatco + VOL02 Ethernet	Chip 0: field; Chip 1: geometry adaptation	Month 5-6
7-board cluster	1000	7 Volatco boards + Ethernet	Full N=1000 hardware replication	Month 12-18

5. Dual-Chip Division of Labor

The dual-chip Volatco configuration (two GA144A12 chips) maps the CQFT slow-fast dynamics onto a natural hardware division:

Chip 0 (field dynamics, fast path):

- Phase and amplitude updates (Kuramoto coupling + noise)
- Memory kernel prediction (dubito term)
- Broadcasts phase to logical neighbors via message queue
- Operates at full GA144 clock rate (≈1 GHz effective)

Chip 1 (geometry adaptation, slow path):

- Receives phase field from Chip 0 every N_SLOW steps
- Computes coherence-weighted routing weight updates
- Returns updated W_ij to Chip 0 routing tables
- Computes Γ proxy and dubito D continuously
- Triggers J7 watchdog if D > threshold ('runaway self-reference')
- Streams observables (R, Γ, D) to host via J8 serial

The λ parameter:

λ = 0: Chip 1 returns constant weights. Geometry frozen. Ablation condition.

λ > 0: Chip 1 returns coherence-modulated weights. Geometry reorganizes.

Toggling λ at runtime switches the entire system between the two regimes in one inter-chip message.

5.1 The λ switch as a containment mechanism

The λ parameter is not merely a tuning knob. It is a hardware-level containment switch grounded in the theoretical framework. A system at λ = 0 cannot undergo directed geometric self-organization, it is structurally confined to the frozen-geometry regime. A system at λ > λ* (estimated λ* ≈ 0.2–0.4 from simulation) is in the self-organizing regime.

Programmatic control of λ from the host (via J8 serial) means the transition between these regimes can be triggered, monitored, and reversed at will. Combined with the dubito watchdog on J7, this gives two independent hardware-level safety mechanisms: a continuous structural monitor (Γ via Chip 1) and a discrete event trigger (D > threshold via J7).

---

6. Specific Questions and Requests for Volatco

6.1 Technical clarifications needed

- What is the maximum reliable multi-hop message latency on the GA144 physical mesh for 4-hop paths? (Needed to estimate geometry-step timing budget.)
- Is SPI flash addressable at the granularity of individual words for routing table lookup? Or is block-level access required? (Affects routing table lookup speed.)
- Does the current polyForth / ArrayForth environment on the evaluation board support runtime modification of inter-core routing (i.e., can a core change its send-port at runtime, not just at compile time)?
- What is the inter-chip communication bandwidth between the two GA144A12 chips on the dual-chip Volatco board? (Needed for Chip 0 → Chip 1 phase field transfer rate.)
- Is the J7 watchdog trigger interface documented for programmatic triggering from a designated core? The CQFT dubito watchdog needs to fire J7 from Chip 1 when D > threshold.

6.2 Collaboration request

The following is proposed as a joint project between DUBITO Inc. and Volatco:

1. DUBITO provides: Penrose patch generator, routing table export tool, Forth pseudocode skeleton (Section 3), simulation predictions for each experimental condition, and data analysis.
2. Volatco provides: GA144 board and polyForth environment, hardware bring-up assistance, J2/J3 power measurement instrumentation, and access to the contractor in Holland for low-level Forth implementation support.
3. Joint output: a hardware paper (targeting ISSCC, NeurIPS hardware track, or DATE conference) reporting the first experimental demonstration of quasiperiodic topology as a performance advantage in neuromorphic computing, grounded in a formal theoretical framework.

The message to Volatco in one sentence:

Your hardware is the only platform in existence whose architecture naturally implements the slow-fast dynamics that our theoretical framework requires, but its native lattice wiring is the one topology our experiments show is counterproductive. A quick software overlay fixes this, and the experiment that validates it would be a publishable result for both projects.

This document is Annex A to: "Geometric Coherence as a Safety Criterion for AGI: A Testable Framework and Neuromorphic Implementation" - Research Proposal, DUBITO Inc., April 2026.

All claims in this document are grounded in peer-reviewable simulation results (N up to 1,000, 20+ runs per condition, 165.7× signal-to-noise ratio on the primary result). Code available on request.

Contact: Daniel Solis, DUBITO Inc., solis@dubito-ergo.com.